MACHINE LEARNING FUNDAMENTALS SERIES

Transformer Neural Networks: Architecture, Workflow & Principles

Deep Dive into Self-Attention, Encoder-Decoder Mechanics, and Demo vs. Paper Comparison

1. Introduction and Problem Motivation

Transformer neural networks represent a revolutionary paradigm shift in natural language processing (NLP) and serve as the foundational architecture for state-of-the-art Large Language Models (LLMs) such as ChatGPT, GPT-4, and BERT. Prior to Transformers, sequence processing relied heavily on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, which processed tokens sequentially across time steps $(t_1, t_2, \ldots, t_N)$. This sequential recurrence created two major bottlenecks:

- **Computational Inefficiency**: Recurrent architectures cannot process words in parallel across GPU cores, resulting in extremely slow training on large corpora.
- **Memory Bottlenecks & Information Loss**: Long-range dependencies and crucial context words (e.g., initial negation words like *"Don't"*) risk getting diluted or lost when compressed into fixed-size hidden states.

Transformers solve both issues by completely replacing recurrence with **Self-Attention Mechanisms**. This allows the model to process all tokens in an input sequence simultaneously in parallel while dynamically scoring the contextual relationship between every pair of words regardless of their distance.

2. Essential Terminology and Key Concepts

To understand the Transformer architecture, several foundational concepts and mathematical components must be defined:

- **Tokens & Vocabulary**: Words, word fragments, or symbols (such as the end-of-sequence token EOS or start-of-sequence token SOS) that make up the model's fixed vocabulary.
- **Word Embeddings**: A trainable lookup network that converts discrete categorical tokens into dense numeric vectors of continuous values ($\mathbb{R}^D$). Tokens with similar semantic meanings or contextual usage are mapped close together in this vector space.
- **Positional Encoding**: Since Transformers process all sequence tokens in parallel without inherent word-order awareness, Positional Encoding injects word-order information by adding a unique sequence of trigonometric values (alternating sine and cosine waves of varying frequencies) to each word's embedding.
- **Queries (Q), Keys (K), and Values (V)**: Internal vector projections derived from position-encoded embeddings:
  - **Query (Q)**: Represents the current token seeking context from other words.
  - **Key (K)**: Represents the address/identity of candidate words being queried.
  - **Value (V)**: Holds the actual semantic content/representation of candidate words.
- **Dot-Product Similarity**: The mathematical inner product calculated between a Query vector (Q_i) and a Key vector (K_j), quantifying how strongly token i should attend to token j.
- **SoftMax Normalization**: Converts raw similarity dot-product scores into a normalized probability distribution ($\sum = 1.0, S_{i,j} \in [0, 1]$), representing the percentage weight assigned to each token.
- **Self-Attention**: A mechanism that calculates contextual relationships between all words within the same input or target sentence, allowing words like *"it"* to correctly link to *"pizza"* rather than *"oven"*.
- **Multi-Head Attention**: Parallel stacks of independent Self-Attention heads, each using distinct trainable weight matrices (W^Q, W^K, W^V) to simultaneously capture different semantic and syntactic relationships.
- **Encoder-Decoder Cross-Attention**: Attentional layers in the Decoder that query the source sentence encodings (K, V from the Encoder, Q from the Decoder) to ensure crucial source words are accurately translated.
- **Residual Connections (Skip Connections)**: Additive bypass pathways that add the input of an

attention block directly to its output ($x + \text{SubLayer}(x)$), preventing vanishing gradients and preserving core identity information across deep layers.

3. Detailed Step-by-Step Operational Workflow

The operational workflow of a Transformer model processing a translation task (e.g., converting the English sentence *"Let's go"* into Spanish *"Vamos"*) follows a two-stage pipeline consisting of the **Encoder** and **Decoder**.

3.1 Phase A: The Encoder Pipeline

The Encoder takes raw input text and outputs a rich, context-aware mathematical representation for every token in parallel:

1. **Word Embedding Lookup**: Each input token (e.g., *"Let's"*, *"go"*) is converted into an embedding vector via a one-hot vector lookup multiplied by trainable weight matrices. Reusing the exact same weight matrix across all positions allows the network to handle sequences of arbitrary lengths.
2. **Positional Encoding Addition**: Trigonometric positional values derived from sine and cosine functions are added directly to the embedding vectors, providing unique positional signatures for each word index.
3. **Self-Attention Query, Key, and Value Generation**: For each token, the position-encoded vector is multiplied by three separate trainable weight matrices (W^Q, W^K, W^V) to generate unique Query (Q), Key (K), and Value (V) vectors simultaneously.
4. **Similarity Scoring via Dot Product**: The dot product between each Query vector and all Key vectors is computed: $\text{Score}_{i,j} = Q_i \cdot K_j^T$.
5. **SoftMax Normalization**: The raw similarity scores are passed through a SoftMax function, producing attention weights that dictate what percentage of each token's representation should be aggregated.
6. **Value Scaling & Aggregation**: Each Value vector (V_j) is multiplied by its corresponding SoftMax weight, and the weighted vectors are summed to produce the final Self-Attention representations.
7. **Residual Connection Addition**: The initial position-encoded vectors are added directly to the Self-Attention outputs, allowing the layer to refine relationships without losing initial word identity.

3.2 Phase B: The Decoder Pipeline & Autoregressive Generation

The Decoder uses the Encoder's representations alongside previously generated target words to produce translated tokens sequentially until an EOS token is emitted:

1. **Initialization**: Decoding begins by feeding a special start/end token (EOS or SOS) into the target language Embedding Layer.
2. **Target Positional Encoding**: Trigonometric positional encodings corresponding to the current output step are added to the target embeddings.
3. **Decoder Self-Attention**: The target representation is processed through a Self-Attention layer (with Query, Key, Value transformations) to maintain relationships among previously generated output words.
4. **Encoder-Decoder Cross-Attention**:
   - The Decoder generates Query vectors (Q_{dec}) from its current state.
   - The Encoder provides Key (K_{enc}) and Value (V_{enc}) vectors from the encoded source sentence.
   - Dot-product similarity scores are computed between Q_{dec} and K_{enc}, passed through SoftMax, and used to scale V_{enc}, ensuring important source words (like negations) are preserved in translation.
5. **Residual Connections**: Bypass connections add the inputs of both decoder attention blocks to their respective outputs.
6. **Linear Projection & Final SoftMax**: The final decoder vectors pass through a Fully Connected Layer with one output unit per token in the target vocabulary, followed by a SoftMax layer to output predicted class probabilities.
7. **Autoregressive Loop**: The token with the highest SoftMax probability (e.g., *"vamos"*) is selected, appended to the target sequence, and fed back into the Decoder in the next iteration until an EOS token is emitted.

4. Comparison: Educational Visual Demo vs. Original Paper

To make the architecture intuitive, pedagogical demonstrations (such as StatQuest's walkthrough) simplify several engineering components compared to the original research paper (*"Attention Is All You Need"* by Vaswani et al., 2017).

Key Differences Between Demo Model and Original Transformer Paper

The visual demonstration simplifies structural hyperparameters to focus on core concepts. Below is a rigorous comparative breakdown:

Architectural Aspect	Simplified Visual Demo (StatQuest)	Original Paper (Vaswani et al., 2017)
Vocabulary Size ($\|V\|$)	Small toy vocabulary (4 tokens: *let's, go, to, EOS*)	Large vocabulary (37,000 sub-word BPE tokens)
Embedding Dim (d_{model})	$d_{\text{model}} = 2$ dimensions per token	$d_{\text{model}} = 512$ dimensions per token
Attention Heads	Single-head attention (1 cell)	Multi-Head Attention ($h = 8$ parallel heads)
Dot-Product Scaling	Raw Dot Product: $Q \cdot K^T$	Scaled Dot Product: $\frac{Q \cdot K^T}{\sqrt{d_k}}$ (prevents vanishing gradients)
Layer Normalization	Omitted for clarity	Applied after every sub-layer ($\text{LayerNorm}(x + \text{SubLayer}(x))$)
Position-wise FFNN	Omitted (Direct residual connection to FC)	2-layer Feed-Forward Network ($d_{\text{ff}} = 2048$, ReLU activation) after attention
Stacked Layers (N_x)	1 Encoder block and 1 Decoder block	$N = 6$ stacked identical Encoder and Decoder layers
Parameter Scale	$\sim$ Tens of trainable weights	$\sim 65,000,000$ to $100,000,000+$ parameters

Table 1: Systematic comparison between pedagogical demo and production Transformer specifications.

5. Mathematical Formulation Summary

The core mathematical operations governing the Transformer architecture are summarized below:

1. **Scaled Dot-Product Attention**:

$$\text{Attention}(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

where d_k is Key vector dimensionality. Division by $\sqrt{d_k}$ prevents large dot products that push SoftMax into vanishing gradient regions.

2. **Multi-Head Attention Assembly**:

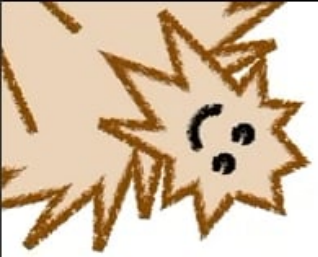
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \quad \text{where head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right) \tag{2}$$

3. **Position-wise Feed-Forward Network**:

$$\text{FFN}(x) = \max\left(0, xW_1 + b_1\right)W_2 + b_2 \tag{3}$$

4. **Trigonometric Positional Encoding**:

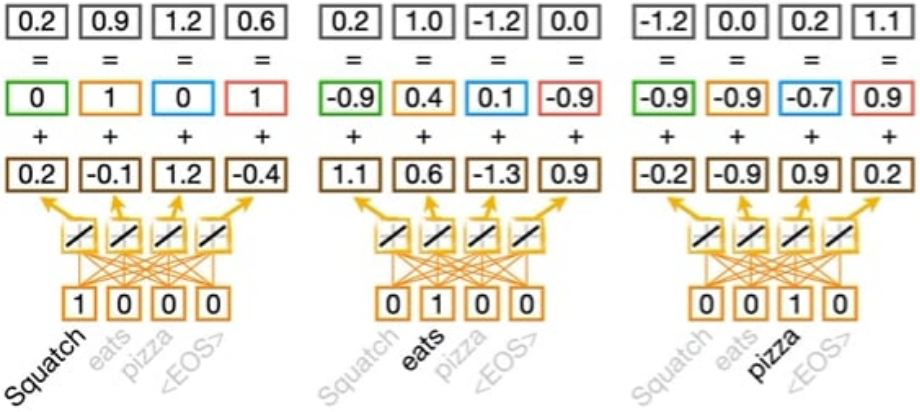
$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \qquad \text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{4}$$

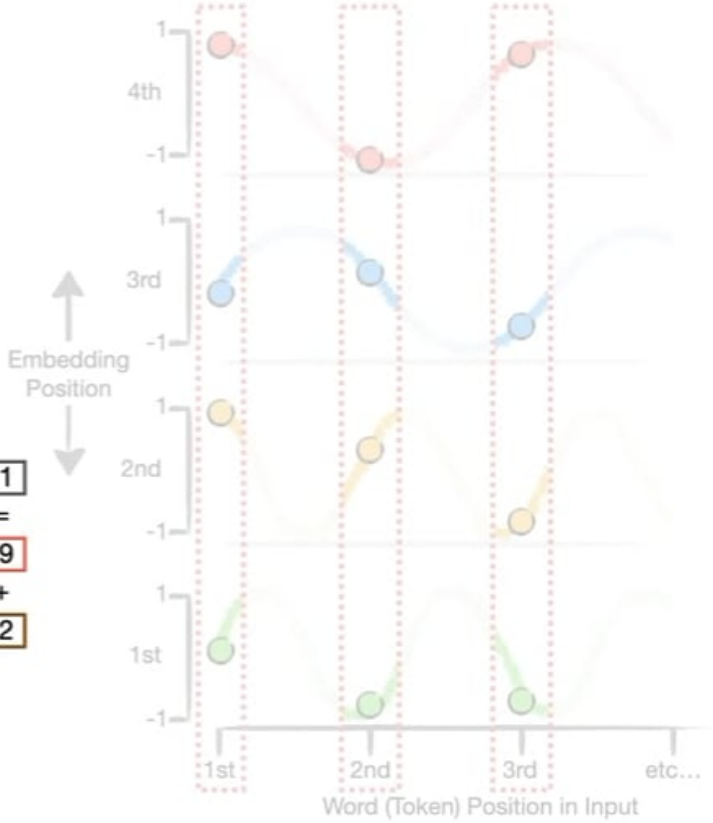


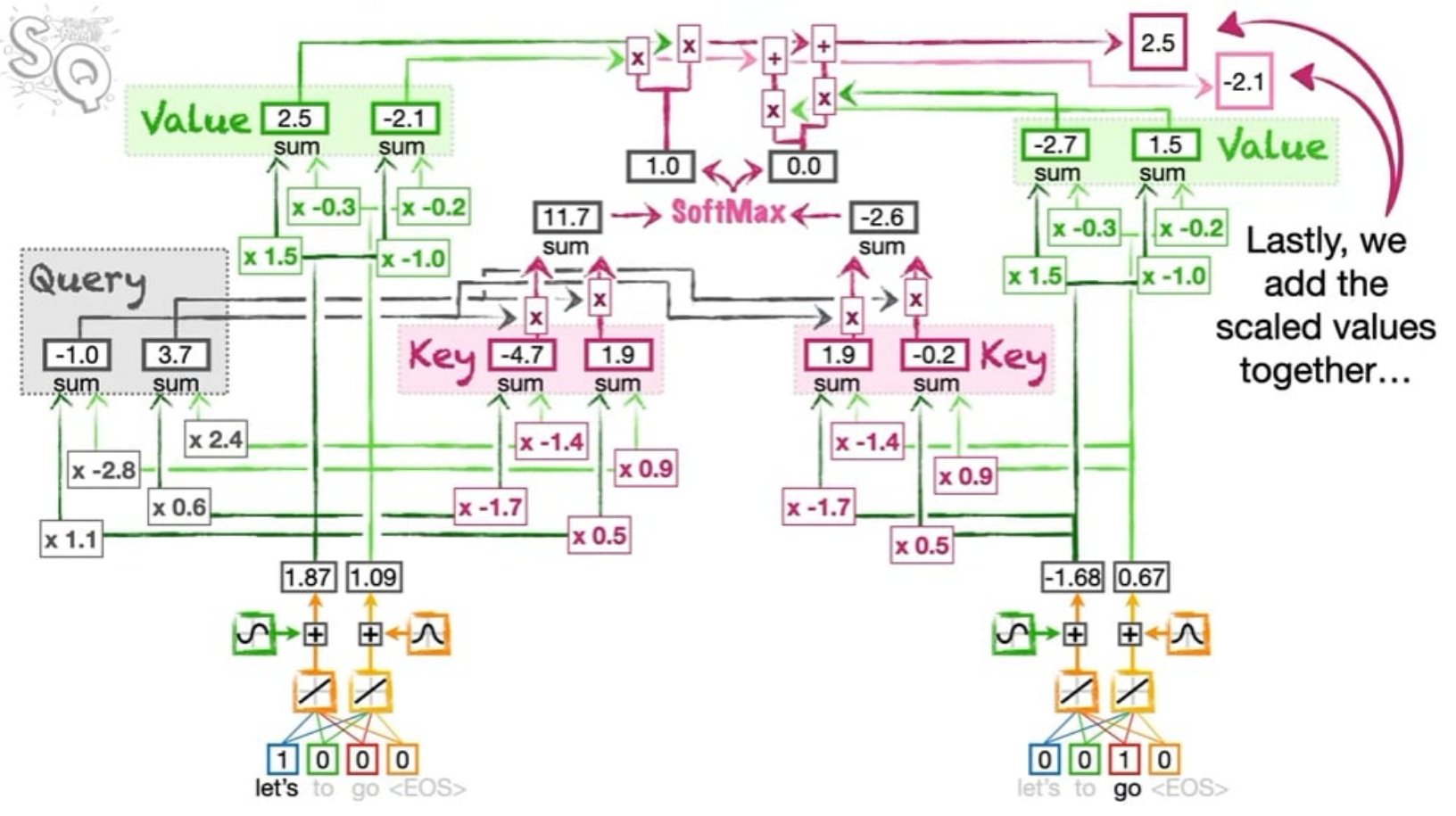
…and we end up with word embeddings plus **Positional Encoding** for the whole sentence…

‘Squatch eats pizza!!!

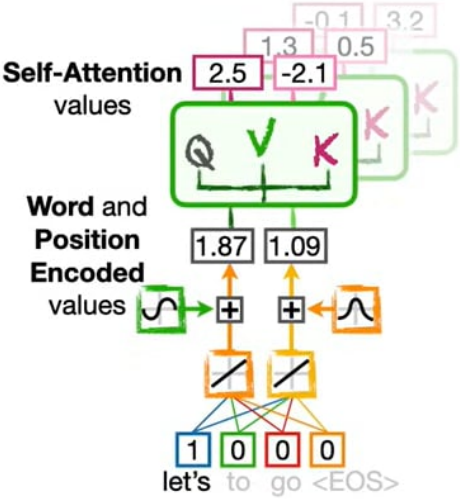


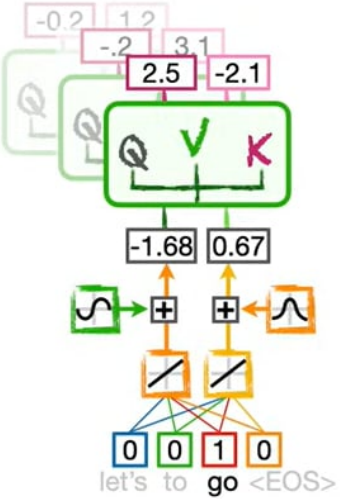




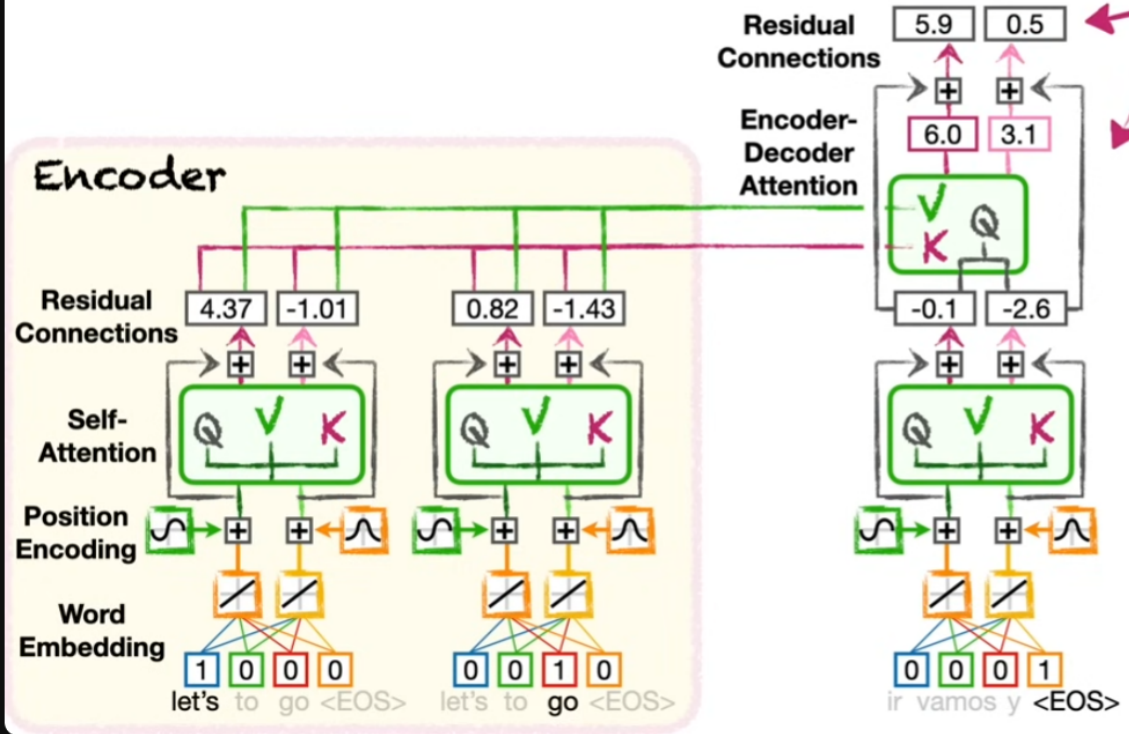

Value
2.5
-2.1
sum
sum
x -0.3
x -0.2
x 1.5
x -1.0
Query
-1.0
3.7
sum
sum
x 2.4
x -2.8
x 0.6
x 1.1
11.7
sum
SoftMax
-2.6
sum
Key
-4.7
1.9
sum
sum
x -1.4
x 0.9
x -1.7
x 0.5
1.0
0.0
2.5
-2.1
-2.7
1.5
sum
sum
Value
x -0.3
x -0.2
x 1.5
x -1.0
1.9
-0.2
Key
sum
sum
x -1.4
x 0.9
x -1.7
x 0.5
1.87
1.09
-1.68
0.67
1 0 0 0
let's to go <EOS>
0 0 1 0
let's to go <EOS>
Lastly, we add the scaled values together…

In the manuscript that first described **Transformers**, they stacked **8 Self-Attention** cells, and they called this **Multi-Head Attention**.





Now we add another set of **Residual Connections**...



...that allow the **Encoder-Decoder Attention** to focus on relationships between the output words and the input without having to preserve the **Self-Attention** or **Word** and **Position Encoding** that happened earlier.

So we run the **2** values through a **Fully Connected Layer** that has one input for each value that represents the current token, so in this case we have **2** inputs…

